EvH – A New Randomized Cipher Paradigm: Deriving Encryption from and with Information Compression and Correction

Hillel Avni[1], Shlomi Dolev[1], Stav Elbar[2], Zvi Galil[3], Komal Kumari[4], Shantanu Sharma[4], Jeffrey Ullman[5], and Moti Yung[2,6]

[1]Ben-Gurion University of the Negev, Israel. [2]Google Inc. [3]Georgia Tech., USA.
[4]New Jersey Institute of Technology, USA. [5]Stanford University, USA. [6]Columbia University, USA.
hillel.avni@gmail.com, dolev@cs.bgu.ac.il, stavelbar@google.com, galil@cc.gatech.edu, kk675@njit.edu, shantanu.sharma@njit.edu, ullman@gmail.com, moti@google.com

Abstract. Standard symmetric encryption schemes, such as AES, other block ciphers and their modes, stream ciphers, etc., are highly effective and efficient for many standard scenarios. All of them have been derived from Shannon's 1948 seminal work on the communication theory of secrecy systems. Here we look at other settings where the situation is somewhat different from the standard one: *e.g.*, while the encryption process may fail to update the ciphertext a limited number of times, or bits of the ciphertext are omitted: can the decryption process nevertheless recover the message in its entirety? Another situation is when encrypting a bulk of messages that should be packed together within the same ciphertext dedicated space (*i.e.*, encryption done holographically on a clockchain space) — can a process compress the messages this way? Another issue is adding further hiding layer to ciphertexts, like hiding the number of messages packed together, or even attempting to hide that encryption (rather than another cryptographic protocol) takes place? Can the new paradigm be based directly on a simple cryptographic (preferably post-quantum) tool? Note that the above scenarios, involving data compression and correction, are naturally derived from Shannon's other 1948 seminal work on Information Theory.

This paper introduces Encryption via Hash (EvH), a new symmetric randomized cipher built upon pseudorandom keyed cryptographic hash (*i.e.*, Message Authentication Code, MAC) functions, and Bloom Filters. EvH's core novelty lies in its prefix decryption capability. This unique property enables a paradigm in which encryption is tightly integrated with online compression and robust resilience to omission errors of the encryption process. By representing message prefixes in a Bloom filter, EvH allows a receiver to decrypt the initial part of a message even if subsequent data are lost, and to recover from some prefix omissions during encryption. At times, these built-in new properties may be significant (even beyond the above examples). Furthermore, this prefix-based approach facilitates simultaneous compression during the decryption phase by dynamically pruning invalid message continuations, using shared μ-gram dictionaries or employing search via Large Language Models (LLMs). The result is a stateless and parallelizable cipher that, while computationally distinct from traditional ciphers, offers unique functional benefits for specific use cases, at the cost of correctness being ensured only probabilistically (as in compression processes), though the error can be well controlled and made significantly small.

Keywords: Bloom Filter · Large Language Model · Symmetric Encryption.

1 Introduction

Symmetric encryption schemes, from classical block ciphers like AES to modern stream ciphers, are designed with a foundational requirement: **deterministic correctness**. The decryption of a valid ciphertext must produce the exact original plaintext, bit for bit. Most bulk encryption settings enjoy the above properties and the speed of optimizations of designs, yet in some settings, the constraints impose significant limitations in certain modern communication over unreliable channels, which are often characterized by data loss and erasure. In such environments, the traditional design may look like over-engineering for speed and correctness but can lead to data loss from even minor transmission errors. For example, transmission of encrypted news/event streams over partially reliable/asynchronous channels, where records may arrive out-of-order or be lost, and here strict block-aligned decryption is undesirable.

Here, we explore a **new paradigm for symmetric encryption**, one in which the strict demand for deterministic correctness is relaxed. We demonstrate that by embracing a controllable probabilistic correctness model, it becomes possible to design ciphers with powerful natively integrated functionalities that are lacking in the traditional deterministic framework. Specifically, we trade a negligible and controllable probability of decryption error for inherent resilience to erasures (*i.e.*, omissions of encryption actions) and the ability to perform compression simultaneously with decryption.

We introduce **Encryption via Hash (EvH)**, a cipher built on this new paradigm. Instead of encrypting discrete blocks of data, EvH's core mechanism involves encoding the entire prefix structure of a message into a single, space-efficient probabilistic data structure — a Bloom filter [4]. A keyed cryptographic hash function, acting as a pseudorandom function (PRF), is used to map each message prefix to a set of positions in the filter. For instance, for a given message "apple," the prefixes will be 'a', 'ap', 'app', 'appl', and 'apple', and they will be inserted into a Bloom filter with the help of a keyed cryptographic hash function. The decryption process is thus transformed from a simple inverse operation into an iterative search for a valid path of prefixes through this structure.

Notations	Meaning
BF	Bloom filter
λ	A security parameter
α	False positive rate for the Bloom filter
σ	Alphabet Size
c	A letter in σ
Δ	A set containing all μ-grams
H	The hash function for Bloom filter
$file$	A file to be stored using EvH
M and m	A message M of size m

Table 1: Frequently used notations in this paper.

For instance, in the context of encrypted news/event stream transmission use-cases, as discussed above, EvH allows the insertion of news/event prefixes into a Bloom Filter and enables decryption even when parts of the transmission are delayed or missing. Since EvH's decryption will be prefix-based (and tolerant to erasures), temporary inconsistencies in record order do not cause catastrophic decryption failure. Once the stream stabilizes, records can be reconstructed in the correct order.

Furthermore, EvH incorporates techniques to improve the efficiency of the decryption process by dynamically pruning invalid message continuations, using shared μ-gram dictionaries, or searching via Large Language Models (LLMs).

Our Contributions

The primary contribution of this work is the introduction and formalization of a new paradigm for symmetric encryption that prioritizes certain functionalities and resilience in unreliable environments over deterministic correctness. Our specific contributions are as follows.

– **A New Cipher Paradigm:** We introduce a novel approach to symmetric cipher design where a controllable, probabilistic correctness model is leveraged to enable powerful unique features.

We show that by relaxing the need for deterministic decryption, functionalities such as inherent erasure tolerance and integrated compression become possible.

- **The EvH Cipher Construction:** We present EvH, the first practical cipher built on this paradigm. EvH works by reducing the problem of message encryption to a private set representation problem, where the set of all message prefixes is encoded into a space-efficient Bloom filter using a PRF. Since we process prefixes, the speed of encryption and decryption is (at least initially– to be improved later) inherently quadratic in the size of the plaintext (so the paradigm gives up on speedy single-pass encryption/ decryption methods).
- **Formal Security Proof:** We prove that the EvH construction achieves IND-CPA security. The proof reduces the security of the cipher to the standard cryptographic assumption of a secure pseudorandom function (PRF).[1]
- **Inherent Erasure Tolerance:** We demonstrate that EvH provides graceful degradation in the face of data loss. By treating lost ciphertext bits as '1's, the receiver can still recover the full message, a significant advantage over traditional ciphers that fail catastrophically.[2] This resilience is particularly critical in distributed or asynchronous environments where the encryption action and data transmission are decoupled. In such scenarios, data loss (action omissions) can occasionally occur, and it is important to be fault-tolerant. EvH ensures that the message remains recoverable, unlike traditional ciphers, where such errors lead to catastrophic failure.
- **Integrated Decryption-Time Compression:** We show how EvH's search-based decryption enables compression to be performed simultaneously with decryption. This is achieved by pruning the search space using shared knowledge, such as μ-gram dictionaries or Large Language Models (LLMs), which allows for smaller ciphertexts — this is, to the best of our knowledge, the first AI-aided cryptographic operation.
- **Advanced Security Properties:** We analyze several advanced properties of EvH, including:
  - **Stateless and Parallelizable Operation:** EvH design is inherently stateless and highly parallelizable (by allowing multiple messages to share a common Bloom filter), making it well-suited for modern distributed systems (*e.g.*, the ciphertext can be deposited inside a blockchain environment with many users accessing).
  - **Message Length Concealment:** EvH requires no padding, eliminating certain attack vectors, and also helps to conceal the true plaintext length (and/or the number of plaintexts).

Outline of the Paper, Appendix, and Code of EvH
§3 defines and proves the security of our core cryptographic primitive, a private set representation. §4 shows how EvH is constructed via a reduction to this primitive. §5 compares EvH against stream and block ciphers. §6 and §7 develop techniques to improve the decryption process efficiency using μ-grams and LLMs, respectively. Appendix provides a formal security proof of EvH. Please see [14] to find the code of EvH.

2 Related Work

Bloom filters and their numerous variants [5] have been widely used in several secure operations, *e.g.*, private set intersection/union, cardinality estimation, and database operations (indexing,

[1] CCA security can, in turn, be obtained using a standard authenticated-encryption (e.g. Encrypt-then-MAC construction): EvH is used as a black-box semantically secure encryption scheme employing the Bloom filter, after which a message authentication code (MAC) is computed over the resulting ciphertext with a second MAC key. The MAC outcome is embedded as a second message into the Bloom filter as well.

[2] For instance, if several bits of multiple blocks are lost of data encrypted using AES-CTR, then the receiver may not recover/decrypt the message, while EvH can do it, due to its built-in erasure tolerance.

conjunctive queries, and join queries). However, a Bloom filter has never been used to solve the problem addressed in this paper.

Bloom filters for private set operations. [19,10,18,2,24,25,22,31] used Bloom filters for private set intersections, where two parties, namely a server and a client, hold their sets of elements, and a client wants to learn the intersection of the sets, without revealing its elements to the server, while the server wants to ensure that the client will only learn the intersection. Two security models have been considered: semi-honest, in which both parties run the protocol as given but wish to learn as much information as possible from the execution, and malicious, in which one or both parties may corrupt the computation. These techniques are based on a homomorphic encryption scheme, *e.g.*, [19], a garbled Bloom filter that uses secret-sharing, *e.g.*, [10,18,2], and a slight variation of a garbled Bloom filter, *e.g.*, [24,25]. [22,31] considered many parties performing PSI, and [22] also considered another variant of PSI, called d-and-over-PSI, where each party learns elements that appear in at least d of the parties. [13] used Bloom filters to compute the cardinality of set-union and set-intersection, and [30] defined a new notion of a virtual Bloom filter to compute the cardinality of PSI. [8] used Bloom filters to compute private set union.

Bloom filters for database operations. Paper [16] was perhaps the first to improve the efficiency of keyword search in a document from linear in document size by using Bloom filters to track keywords that appear in documents. [1] also used Bloom filters for encrypted search. [7] also used Bloom filters to securely find all documents containing a query keyword in a document database, while providing additional security via two non-colluding clouds. [21] created an indistinguishable Bloom filter-based tree to securely execute conjunctive searches (*i.e.*, all records of a table having 'a' and 'b'), and [20] also deals with conjunctive queries. [29,15] used Bloom filters for a secure fuzzy/similarity search. Other applications of Bloom filters are secure join of two tables [6] and secure record linkage/entity resolution [11,27].

Encryption via pseudorandom MAC. To clarify, we do not claim to be the first ones to employ MAC or pseudorandom one-way functions for encryption. Obviously, theoretically any one-way function can be transformed into a pseudorandom permutation via a known reduction in elaborate constructions. Also, we do not mean to compete with the performance optimizations of block ciphers achieved over many years of important research. In addition, chaffing and winnowing (CW) by Rivest [26] shows how covert channels can be used for encryption via pseudorandom MAC used for authentication (and rejected sampling). Rather, EvH's goals as a new paradigm used directly by combining a pseudorandom MAC and Bloom filter is what we claim as an interesting new setting (with interesting properties).

3 A Primitive for Private Set Representation

Before detailing the EvH cipher, we first establish its core cryptographic building block: a primitive for representing a private set of elements. The goal of this primitive is to create a data structure representing a secret set S in a way that allows for membership queries but reveals nothing about the elements of S beyond the results of these queries.

Definition 1 (Private Set Representation). *A Private Set Representation (PSet) is a collection of three polynomial-time algorithms* $(\mathsf{Setup}, \mathsf{Insert}, \mathsf{Check})$*:*

- $\mathsf{Setup}(1^\lambda) \rightarrow K$*: A probabilistic algorithm that takes a security parameter* λ *and outputs a secret key* K*.*

- $\mathsf{Insert}(K, x) \to D_x$*: A deterministic algorithm that takes the key* K *and an element* $x \in \{0,1\}^*$*, and outputs a representation* D_x *for that single element. The complete data structure* D_S *for a set* $S = \{x_1, \ldots, x_n\}$ *is the collection* $\{D_{x_1}, \ldots, D_{x_n}\}$*.*
- $\mathsf{Check}(K, x, D_S) \to \{0,1\}$*: A deterministic algorithm that takes the key* K*, an element* x*, and a set representation* D_S*. It outputs 1 if* $x \in S$ *and 0 otherwise.*

The security of this primitive ensures that the data structure D_S does not leak information about the elements within the set S. We formalize this using a standard indistinguishability game.

Definition 2 (IND-PSet Security). *A PSet scheme is Indistinguishable for Private Sets (IND-PSet) secure if for any probabilistic polynomial-time (PPT) adversary* $\mathcal{A}$*, its advantage in the following game is negligible in the security parameter* λ*:*

1. ***Challenge:*** *The adversary* $\mathcal{A}$ *produces a pair of sets,* (S_0, S_1)*, of equal size, i.e.,* $|S_0| = |S_1|$*.*
2. ***Setup:*** *The challenger generates a secret key* $\mathsf{Setup}(1^\lambda) \to K$ *and chooses a random bit* $b \in \{0,1\}$*.*
3. ***Computation:*** *The challenger computes the data structure* D_b *by running* $\mathsf{Insert}(K, x)$ *for all* $x \in S_b$*.*
4. ***Response:*** *The challenger sends the resulting data structure* D_b *to the adversary* $\mathcal{A}$*.*
5. ***Guess:*** $\mathcal{A}$ *outputs a bit* b'*.*

The advantage of the adversary is defined as

$$Adv_{\mathcal{A}}^{\textit{IND-PSet}}(\lambda) = |\Pr[b' = b] - 1/2|.$$

3.1 A Secure PRF-based Construction

We can construct a secure PSet primitive from any secure pseudorandom function (PRF) family.

Construction. Let $\mathcal{F} = \{F_K : \{0,1\}^* \to \{0,1\}^\lambda\}_{K \in \{0,1\}^\lambda}$ be a PRF family.

- $\mathsf{Setup}(1^\lambda)$: Choose a key K for the PRF uniformly at random from $\{0,1\}^\lambda$.
- $\mathsf{insert}(K, x)$: The representation for an element x is simply $F_K(x)$. For a set S, the data structure D_S is the multiset $\{F_K(x) \mid x \in S\}$.
- $\mathsf{check}(K, x, D_S)$: Output 1 if and only if $F_K(x) \in D_S$.

Theorem 1. *If* $\mathcal{F}$ *is a secure PRF family, then the construction above is an IND-PSet secure private set representation.*

Proof. The proof is a standard reduction to the security of the PRF. Assume for contradiction that there exists a PPT adversary $\mathcal{A}$ that can break the IND-PSet security of the construction with a non-negligible advantage $\epsilon(\lambda)$. We will construct a distinguisher $\mathcal{D}$ that uses $\mathcal{A}$ as a subroutine to break the PRF security of $\mathcal{F}$ with the same advantage.

The distinguisher $\mathcal{D}$ is given access to an oracle $\mathcal{O}$ which is either a real PRF F_K (for a randomly chosen unknown key K) or a truly random function f. $\mathcal{D}$'s goal is to determine which case it is. $\mathcal{D}$ simulates the IND-PSet game for $\mathcal{A}$:

1. $\mathcal{A}$ sends its challenge sets (S_0, S_1) to $\mathcal{D}$.
2. $\mathcal{D}$ chooses a random bit $b \in \{0,1\}$. It then constructs the challenge data structure D_b by querying its own oracle $\mathcal{O}$ for every element $x \in S_b$. That is, $D_b = \{\mathcal{O}(x) \mid x \in S_b\}$.
3. $\mathcal{D}$ sends D_b to $\mathcal{A}$.
4. $\mathcal{A}$ outputs its guess b'. If $b' = b$, $\mathcal{D}$ guesses that its oracle was the real PRF and outputs 1. Otherwise, it outputs 0.

Now, let us analyze the advantage of our distinguisher $\mathcal{D}$.

- **Case 1: The oracle is the real PRF ($\mathcal{O} = F_K$).** In this scenario, $\mathcal{D}$ provides $\mathcal{A}$ with a perfectly simulated environment of the real IND-PSet game. By our assumption, $\mathcal{A}$ has an advantage ϵ. The probability that $\mathcal{A}$'s guess is correct is $\Pr[b' = b] = 1/2 + \epsilon$. Therefore, $\Pr[\mathcal{D}(F_K) = 1] = 1/2 + \epsilon$.
- **Case 2: The oracle is a truly random function ($\mathcal{O} = f$).** The sets S_0 and S_1 differ in at least one element. For any $x \in (S_0 \setminus S_1) \cup (S_1 \setminus S_0)$, the value $f(x)$ is a fresh, uniformly random value. From the adversary's perspective, both data structures $D_0 = \{f(x) \mid x \in S_0\}$ and $D_1 = \{f(x) \mid x \in S_1\}$ are just collections of uniformly random and independent values. They are therefore computationally indistinguishable. $\mathcal{A}$ has no information to leverage and its advantage is negligible. Thus, $\Pr[b' = b] \approx 1/2$. Therefore, $\Pr[\mathcal{D}(f) = 1] \approx 1/2$.

The advantage of the distinguisher $\mathcal{D}$ in breaking the PRF security of $\mathcal{F}$ is:

$$Adv^{\text{PRF}}_{\mathcal{D}}(\lambda) = |\Pr[\mathcal{D}(F_K) = 1] - \Pr[\mathcal{D}(f) = 1]| \approx |(1/2 + \epsilon) - 1/2| = \epsilon$$

We have shown that if a PPT adversary $\mathcal{A}$ can break the PSet construction with non-negligible advantage ϵ, then we can construct a PPT distinguisher $\mathcal{D}$ that breaks the underlying PRF security with the same advantage. This contradicts our initial assumption that $\mathcal{F}$ is a secure PRF. Therefore, the PSet construction is IND-PSet secure.

Note on Efficient Implementation. The construction above, where the data structure D_S is a simple multiset of PRF outputs, is conceptually simple but not space-efficient. In a practical system, storing the full λ-bit output for every element in the set can be costly. For this reason, the EvH cipher, which we will describe next, uses a **Bloom filter** as a space-efficient, probabilistic implementation of this PSet primitive. The Bloom filter stores a compressed representation of the set of PRF outputs, trading a small, controllable probability of false positives during the Check operation for a significant reduction in space.

4 EvH: Encryption via Hash

Having formally defined the PSet primitive, we now detail the construction of EvH cipher. The core idea is a reduction from the problem of encrypting a message M to PSet problem defined in §3.

Specifically, the secret set that we represent is the set of all prefixes of the message M. The most direct way to encode this prefix structure is to generate a set, ***P***, containing the output of a PRF (*e.g.*, a keyed SHA256 that produces hashes of all prefixes). For example, to encrypt the string "FUN," one would create a set of the hashes of prefixes: "F," "FU," and "FUN." While simple, this approach incurs unacceptable space overhead. Assuming a 32-byte hash output (like SHA-256), storing the prefixes for this three-character message would require 96 bytes.

To overcome this, EvH uses a **Bloom filter** — a space-efficient probabilistic data structure used to store a compressed representation of the PRF outputs for each prefix and to test whether an element is a member of the secret set, as discussed above. By replacing the PRF output set (*e.g.*, full hash digests) with a Bloom filter, we significantly reduce space usage at the cost of handling a controllable rate of false positives during decryption.

Note. ***Whenever we write a hash function H in this paper, we mean a keyed pseudorandom function $H_K(\cdot)$, instantiated for example using HMAC-SHA256***.

4.1 Encrypting a Message into a Bloom Filter

The encryption process begins with a shared secret key, K, and a randomly generated Initialization Vector (IV), typically 256–512 bits long, to ensure semantic security. For a given message

M, the sender applies the PRF (*e.g.*, a keyed SHA256) to every prefix of the concatenated string $IV||M$. Each resulting PRF output is then inserted into a Bloom filter (BF), which constitutes the final ciphertext.

This construction, in which the message is represented by its nested prefix hashes, is the foundation for EvH's unique properties. The final Bloom filter itself is the ciphertext sent to the receiver. Later, we also discuss how to deal with messages of different sizes.

Example 4.1. To encrypt the message "apple is red" with the key K and a random IV i, the sender would compute and insert the following sequence of PRF outputs (*e.g.*, hash digests) into the Bloom filter:

$$\text{PRF}(K, i||\text{a}), \text{PRF}(K, i||\text{ap}), \text{PRF}(K, i||\text{app}), \text{PRF}(K, i||\text{appl}), \ldots, \text{PRF}(K, i||\text{apple is red})$$

The resultant Bloom filter is sent to the receiver. □

Algorithm 1 description. Algorithm 1 provides pseudocode of the encryption process. ***For simplicity, in the algorithm's pseudocode, we do not use the secret key*** K ***and the IV***. For our purposes, the hash function H is instantiated as a SHA-256 hash object, providing the following two methods/operations for the encryption algorithm:

1. update($data$) that feeds the given data and extends the hash to incorporate the new input while preserving all previously processed prefixes.
2. digest() that outputs the hash value corresponding to the current internal state, *i.e.*, the hash of the entire input sequence accumulated through all preceding update() calls.[3]

Algorithm 1 first computes the size of the file in Line 2 and initializes a Bloom filter with the computed size and a specified false positive probability in Line 3. Subsequently, the algorithm processes each newly received character by extending all previously seen prefixes using the update() method of the hash function (Line 8), then computes the corresponding digest(), which is then used to set a bit of the Bloom filter (Line 6).

Once the entire message is inserted into the Bloom filter, the algorithm appends a fixed-length postamble/suffix to the Bloom filter using the same procedure, Line 8. As will be clear soon in the decryption process, this postamble/suffix ensures that the receiver can recover the complete and correct message in cleartext.

Furthermore, if the IV has not been negotiated between the sender and the receiver for a message, the IV can also be encrypted into the Bloom filter using the secret key, followed by the message. Without knowing the IV, the receiver cannot decrypt the message.

Note. SHA-256 is incrementally computable: given the hash state of a prefix P, computing the hash of $P + c$ will cost $O(1)$. Thus, EvH PRF evaluations grow linearly with message length. Also, the following subsection develops an efficient method for encrypting a message using a directed acyclic graph.

An Optimization: Prefix Hashing to "Proper Continuation" Hashing. Now, we can imagine a cleartext message as a directed acyclic graph (DAG), and the encryption process becomes a breadth-first search (BFS) traversal of this graph by encoding paths into a Bloom filter, an edge at a time. This will make the encryption process a single linear pass over the message.

[3] Consider a string "FUN." Initially, the hash object H is empty. After executing update(F), the internal state represents the hash of the prefix "F," and calling digest() returns hash digest, *i.e.*, H(F). Extending the state with update(U) updates the internal state to represent the prefix "FU," and digest() now returns H(FU). Note that if one computes H(F), H(FU), and H(FUN) independently without using **update**, the hash of the longer prefix must be computed from scratch, which increases the computational cost linearly with the prefix length.

Algorithm 1: Encrypt a message/file into a Bloom Filter

```
1  Function EncryptFile(file, H)
2  size ← len(file)
3  BF ← BloomFilter(size, prob)
4  for i ← 0 to size − 1 do
5  |   H.update(file[i])
6  |   BF.insert(H.digest())
7  foreach φ ∈ postamble do
8  |   H.update(φ),
   |     BF.insert(H.digest())
9  return BF;
```

Algorithm 2: Decrypt a message/file from a Bloom Filter

```
1  Function DecryptFile(BF, H, size, postamble)
2  P ← ⊥, Q_bfs ← ⊥ ▷ Start from an empty prefix and creating a queue
3  Q_bfs.append(⟨H, P⟩)      ▷ Initialize BFS queue with the empty string
4  while Q_bfs ≠ ⊥ do
5  |   ⟨H, P⟩ ← Q_bfs.pop ▷ Dequeue a pair of hash function & a prefix
6  |   if len(P) < size then
7  |   |   foreach c ∈ σ do
8  |   |   |   P_opt ← P+c, H_opt ← H.copy(), H_opt.update(c)
9  |   |   if BF.check(H_opt.digest()) then
10 |   |   |   Q_bfs.append(⟨H_opt, P_opt⟩)   ▷ Enqueue candidate pair
11 else
12 |   success ← true
13 |   foreach φ ∈ postamble do
14 |   |   H.update(φ)
15 |   |   if not BF.check(H.digest()) then
16 |   |   |   success ← false & break ▷ False positive full message
17 |   if success = true then
18 |   |   return P
19 return ⊥;
```

Let $M = x_1x_2 \dots x_m$ be a cleartext. We construct a DAG (i.e., path graph in this case but it can also be generalized), where each character $x_i \in M$ is represented by a vertex v_i of the graph and each directed edge e_i is a tuple of the form

$$e_i = \langle i, i+1, x_{i+1} \rangle \mid i \in \{1, \dots, m\}\rangle$$

e_i represents a transition from position i to $i+1$, labeled with the character x_i. Additionally, there is a start vertex $start$, with an edge connecting it to the first character:

$$\langle start, 1, x_1 \rangle$$

For example, a message "FUN," whose edges will be as follows:

$$\langle s, 1, \mathrm{F}\rangle, \quad \langle 1, 2, \mathrm{U}\rangle, \quad \langle 2, 3, N\rangle.$$

Now, we encrypt each edge independently, as $PRF(K||\mathrm{IV}, \langle i, i+1, x_{i+1}\rangle)$ and insert the resulting value into a Bloom filter.

4.2 Decrypting a Message from a Bloom Filter

Decryption in EvH is fundamentally a search process. The receiver, possessing the shared key K, the cleartext IV (either prenegotiated for the message or retrived from the received Bloom filter itself), and the received Bloom filter BF, must reconstruct the original message by finding the valid path of prefixes that were encoded by the sender. The search is typically performed as a Breadth-First Search (BFS) over a tree of possible prefixes, starting from the known IV.

For each valid prefix found, the receiver appends every character c from the alphabet σ to form new candidate prefixes. For each candidate, it computes the corresponding PRF/hash output and checks whether it is present in the Bloom filter. If the check returns true, the new candidate prefix is added to the queue for the next iteration; otherwise, that path is pruned. This process continues until a path reaches the known length of the original message. To ensure correctness against full-message false positives, a known postamble is appended before encryption and verified after decryption.

Example 4.2. Suppose the receiver obtains the Bloom filter corresponding to the encryption of the message "apple is red," as described in Example 4.1. The receiver knows the secret-key K, IV i, and the length of the original message.[4]

Decryption proceeds as a breadth-first search over prefixes, starting from the IV. The receiver first computes PRF$(K, i||c)$ for every character $c \in \sigma$, where σ is the set of alphabet, and checks each value against the Bloom filter. Suppose, only PRF$(K, i||\text{a})$ is found in the Bloom filter BF, so the prefix "a" is added to the queue. Next, the receiver extends this prefix by adding each character $c \in \sigma$ and checks PRF$(K, i||\text{aa})$, PRF$(K, i||\text{ab}), \ldots$ in the Bloom filter. Suppose, only PRF$(K, i||\text{ap})$ is present in the Bloom filter BF, and thus the prefix "ap" is added to the queue. This process continues iteratively to decrypt the entire message.[5] □

Algorithm 2 description. Algorithm 2 provides pseudocode for the decryption process. ***For simplicity, in the pseudocode, we do not use the secret key*** K ***or the IV during decryption***.

In addition to `update()` and `digest()` methods used in Algorithm 1, the decryption algorithm uses a `copy()` method that creates a copy of the current hash object to preserve the hash state of the existing prefix while allowing new characters to be appended without affecting other candidate prefixes. This is crucial for exploring multiple possible extensions of a prefix (in parallel).

The algorithm assumes that the size of the original message is known. It initializes an empty prefix P and a queue Q_{bfs} containing a pair of the empty prefix and an initial empty hash object, Line 2-Line 3.

Then, the algorithm proceeds iteratively until the queue Q_{bfs} becomes empty. At each iteration, the algorithm dequeues a pair $\langle H, P \rangle$, Line 5, where H is the current hash state and P is the associated prefix. If the length of P is less than the message size, the algorithm generates new candidate prefixes, denoted by P_{opt}, by appending each character/symbol c from the alphabet σ, Line 8. For each candidate prefix, the algorithm creates a copy of the hash object using `copy()` (Line 8), updates it with c (Line 8), and checks whether the resulting digest exists in the Bloom filter, Line 9. If the candidate is present in the Bloom filter, the current hash object and the prefix are inserted into the queue for further exploration, Line 10.

Once a prefix reaches the full message size, the algorithm performs a final verification using the postamble/suffix, since the Bloom filter may produce false positives, *i.e.*, some prefix P (or the message) may appear valid but is not the original message. The algorithm checks each postamble character in the Bloom filter, Line 13-Line 16. If all postamble characters are present, the algorithm returns the reconstructed message P (Line 18). If the bit of the Bloom filter for the digest of P with a postamble character is not present, it means this candidate prefix P is not the real message.

4.3 Correctness of EvH

The correctness of the EvH decryption process relies on the property of Bloom filters that they allow no false negatives. Consequently, if a message prefix was inserted during encryption, the `Check` operation will always return `true`. This guarantees that the correct path in the prefix tree is never pruned.

However, since Bloom filters permit false positives with probability α, the decryption algorithm may explore invalid paths. For the scheme to be correct in a practical sense (*i.e.*, feasible),

[4] The length of a message can also be encrypted in a Bloom filter, like the message, and then the receiver can also learn the message length. To encrypt the message length ℓ, which is unknown to the receiver, the sender can append a long tail of postamble/suffix, e.g., 'zzz...'. If sufficient zs are recovered, the prefix will indicate message length.

[5] Finally, the receiver appends the known postamble (refer to Line 7 of Algorithm 1) and verifies that the corresponding PRF outputs are also present in the Bloom filter, to ensure the correct and complete message has been decrypted.

the number of false positive paths must remain bounded and not grow exponentially with the message length m.

Let σ denote the alphabet size and let $F(k)$ be the expected number of false positive candidate prefixes at step k (prefixes of length k). At step $k = 0$, we start with the known IV, so $F(0) = 0$. At any step k, the algorithm extends the single true prefix and the $F(k)$ false prefixes by all σ characters. The single true prefix generates exactly one true continuation. The remaining $\sigma - 1$ incorrect extensions of the true prefix, as well as all σ extensions of the $F(k)$ false prefixes, are checked against the Bloom filter. Thus, the expected number of new false positives at step $k + 1$ is given by the recurrence:

$$F(k+1) = \alpha \cdot [(\sigma - 1) + F(k) \cdot \sigma] \approx \alpha \cdot \sigma \cdot (F(k) + 1) \tag{1}$$

Let $r = \alpha \cdot \sigma$. The recurrence simplifies to $F(k+1) = rF(k) + r$. Solving this linear recurrence with the initial condition $F(0) = 0$ yields the closed-form solution:

$$F(k) = \frac{r}{1-r}(r^k - 1) \tag{2}$$

This result establishes the critical condition for the correctness and feasibility of EvH:

1. **Case** $r \geq 1$ **(Infeasible):** If $\alpha \cdot \sigma \geq 1$, the term r^k grows exponentially. The decryption queue explodes, rendering the process computationally intractable for long messages.
2. **Case** $r < 1$ **(Feasible):** If $\alpha \cdot \sigma < 1$, the term r^k decays to 0 as k increases. The number of false positive candidates stabilizes to a constant:

$$\lim_{k \to \infty} F(k) = \frac{r}{1-r} \tag{3}$$

Therefore, provided that the system parameters are chosen such that $\alpha < 1/\sigma$, the expected number of false positives at any step is constant, and the total complexity of decrypting a message of length m is linear, $O(m)$.

Finally, to ensure unique decodability among the surviving paths at step m, the postamble (as described in Algorithm 1) serves as a verification suffix. Since the probability of a false path matching both the message length and the specific Postamble hashes is negligible ($\approx \alpha^{\text{len(postamble)}}$), the receiver can uniquely identify the correct message.

4.4 Experimental Results

In this subsection, we experimentally validate the analytical false-positive model, given in §4.3, and study its implications in practical settings. We investigate:

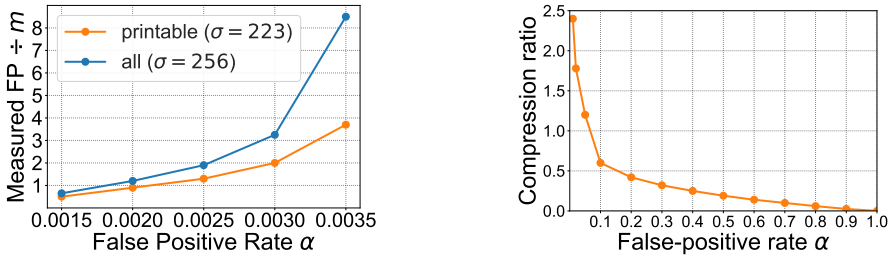
1. The growth of false positives during decryption on varying false positive rate α, in Experiment 4.4.1.
2. The space efficiency of the Bloom filter–based ciphertext representation, in Experiment 4.4.2.

For experiments, we downloaded 200MB of plaintext from Wikipedia. For these experiments, the alphabet size is fixed to $\sigma = 256$ (all ASCII code) and the Bloom filter false-positive rate α is varied.

Let M be a plaintext message of length by m (bytes). Note that since EvH encrypts each prefix into the Bloom filter, the number of true prefixes is also m.

Experiment 4.4.1: False-positive growth during decryption. This experiment measures how the number of false positives changes as the false positive rate α varies.

Theoretical prediction. From §4.3, when $\alpha \cdot \sigma < 1$, the expected total number of false positives grows linearly with the message length. From Eq 3, we have $(\alpha \cdot \sigma)/(1 - \alpha \cdot \sigma)$.



(a) Exp 4.4.1: False-positive growth at decryption. (b) Exp 4.4.2: Space efficiency vs false positive rate.

Fig. 1: Impact of false positive rate α in EvH..

For $\alpha = 0.0035$ and $\sigma = 256$, we have $\alpha \cdot \sigma = 0.896 < 1$, and the predicted number of false positive is: $(0.0035 \cdot 256)/(1 - 0.0035 \cdot 256) = 8.61$.

Experimental result. Figure 1a shows the measured false-positive overhead as a function of the Bloom filter false-positive rate α. The x-axis represents the Bloom filter false-positive probability α, and the y-axis represents the normalized false-positive overhead that is defined as the total number of candidate prefixes explored during decryption, minus the m true prefixes corresponding to the actual message, and normalized by the message length m, *i.e.*,

$$\text{Measured FP} = (\#\text{candidate prefixes explored} - m)/m.$$

In other words, Measured FP will capture the average number of false-positive prefixes processed per message byte across the entire 200MB Wikipedia data.

In Figure 1a, the measured false-positive overhead is approximately 8.6 when $\alpha = 0.0035$, and this matches the experimental results with the theoretical prediction. In Figure 1a, the printable alphabet refers to the subset of ASCII characters that are human-readable. Restricting decryption to printable alphabet reduces $\sigma = 223$, which in turn lowers the effective growth rate $\alpha \cdot \sigma$ and results in the smaller false-positive overhead.

When α is increased to 0.004, we obtain $\alpha \cdot \sigma = 1.024 > 1$, making decryption infeasible, as has been discussed in §4.3.

Experiment 4.4.2: Space efficiency vs false positive rate. Figure 1b shows the space efficiency of EvH on varying false positive rate α. The x-axis shows the Bloom filter false-positive rate α, and the y-axis shows the ***compression ratio*** that is defined as the ratio between the Bloom filter size and the raw message size, measured in bytes:

$$\text{Compression ratio} = \text{Bloom filter size (in bytes)}/\text{Raw message size (in bytes)}$$

In Figure 1b, as α increases, the required Bloom filter size decreases, showing improved compression. When $\alpha \approx 0.05$, the ratio is close to 1, indicating no compression. When $\alpha \approx 0.1$, the ratio drops to approximately 0.5, demonstrating that EvH can achieve significant space savings. However, this apparent space efficiency must be interpreted jointly with the false-positive behavior shown in Figure 1a.

Recall that while increasing α reduces the Bloom filter size by allowing a higher false-positive probability, it simultaneously increases the number of spurious prefixes, making EvH decryption infeasible due to $\alpha \cdot \sigma > 1$, for $\sigma = 256$.

Consequently, although Figure 1b indicates that large values of α produces a better compression ratio, these α values will make EvH decryption impossible. Thus, for $\alpha = 0.0035$, we have 1.5 times memory overhead. In other words, with $\sigma = 256$ and the constraint $\alpha \cdot \sigma < 1$, EvH does *not* achieve compression—the Bloom filter is larger than the original message. §6 and §7 will develop techniques to achieve compression.

4.5 Complexity Analysis

Analysis of EvH's performance is essential to understand its role and practical applications.

Computational Overhead. The primary source of computational cost in EvH is the invocation of the underlying PRF (*e.g.*, a secure hash) function. During encryption, the process requires n PRF/hash operations for a message of n characters (or segments). The decryption process is more intensive, requiring at most $n \cdot \sigma$ hash operations in the worst case, where σ is the size of the alphabet being searched at each step. This contrasts sharply with the efficiency of stream ciphers or AES in a standard mode like CTR, which requires only $O(n)$ block operations. This higher computational cost is the explicit trade-off for gaining the unique functionalities of EvH that will be discussed in detail in §5.2, such as inherent erasure tolerance and integrated compression capabilities, which are not offered by the faster alternatives, such as AES.

Efficiency gain via parallel operation: It is crucial to note that the EvH decryption process is highly parallelizable. The search across the alphabet σ for each prefix extension consists of independent computational paths, see Line 8 of Algorithm 2. On modern multi-core architectures, these paths can be explored in parallel, significantly mitigating the practical impact of the $n \cdot \sigma$ worst-case complexity.

Aside. We develop techniques to further improve the efficiency of the decryption process using μ-grams and LLMs in §6 and §7.

5 EvH and Other Encryption Techniques

In this section, we discuss unique advantages offered by EvH compared to stream and block cipher techniques.

5.1 EvH vs Alternative Hash-Based Encryption

It is essential to compare EvH against simpler hash-based constructions, such as using a PRF as a stream cipher (*e.g.*, $c = (r, \text{PRF}(K, r) \oplus m)$, where K is a secret key and r is a random nonce). While secure and efficient, this standard construction does not offer the specific functional advantages provided by EvH.

Below, we discuss four advantages of EvH that are not offered by stream ciphers, as follows:

– **Erasure Tolerance.** In the stream cipher, any lost bits in the ciphertext result in irrecoverable loss of the corresponding plaintext bit. In contrast, a key advantage of EvH is its inherent resilience to erasures that occur when parts of the ciphertext (*i.e.*, bits of the Bloom filter) are lost or corrupted during transmission, offering graceful degradation in EvH.
Mechanism. A receiver in EvH can handle erasures gracefully. Any missing or ambiguous bit in the received Bloom filter is treated as a '1' during check() operation of Algorithm 2, Line 9. While this conservative approach increases the number of false positives, it guarantees that the true decryption path is never prematurely pruned, allowing the original message to be recovered as long as a sufficient portion of the filter remains intact.
Performance trade-off: The resilience of EvH comes at a predictable cost. The number of false positive paths that the decryption algorithm must explore is proportional to α (the false positive rate) and σ (the alphabet size). Treating erased bits as '1's effectively increases α for the affected regions of the filter, leading to a higher computational complexity for the decryption search. EvH's parameter tuning can play a role in the tradeoff between resilience and performance.
– **Parallelizable Operation:** EvH construction is highly parallelizable. A large message can be split into chunks and encrypted in parallel. More importantly, the decryption search across the alphabet σ for each prefix can be parallelized, significantly mitigating the practical impact of

the worst-case complexity on modern multi-core architectures. Furthermore, EvH can enable parallel encryption of streaming environments where multiple processes concurrently encrypt independent messages into a shared Bloom Filter. The receiver, knowing the IVs associated with each message, can independently recover each message. However, traditional symmetric encryption requires strict message framing and separation to avoid message ambiguity.
This property is important when multiple messages share a common prefix and the same IV is used, EvH naturally compresses the repeated prefixes in a Bloom filter, reducing communication overhead. Trade-off depends on IV management—using distinct IVs per messages preserve maximum security, while a shared IV for prefix-equal messages can improve efficiency with minimal risk if managed carefully. Furthermore, this property enhances privacy by concealing metadata: an adversary cannot determine the exact number of messages contained within a given ciphertext, only the total size of the Bloom filter.[6]

– **Stateless Operation:** An important design principle of EvH is its statelessness. Unlike stateful schemes such as stream ciphers, which require careful management of nonces or counters across sessions, EvH requires no synchronization beyond the shared key. In EvH, any randomness can be generated locally (based on pseudorandom sequences derived from a shared key) and included as a message prefix without requiring coordination or maintaining previously used nonces. Consequently, EvH avoids the need to manage nonces.
This property is highly desirable in modern distributed systems and aligns with the design of other modern cryptographic primitives, such as stateless hash-based signatures [3,9], where avoiding state management is crucial for robustness and security.
– **Integrated Compression via Search Pruning:** While stream ciphers such as AES-CTR require a separate compress-then-encrypt step to reduce the amount of data transmitted, EvH does compression by allowing the transmission of a highly compact, probabilistic ciphertext (a dense Bloom filter). During decryption, the receiver incrementally reconstructs the plaintext by exploring candidate paths and pruning invalid ones. This effectively fuses decompression and decryption into a single operation, enabling the streaming of compressed data without explicit pre-compression algorithms. §6 and §7 develop methods that leverage shared knowledge, such as k-gram dictionaries or language models, to efficiently prune invalid paths during decryption.

5.2 EvH vs Traditional Block Ciphers

EvH offers the following powerful features that are not natively supported by traditional ciphers:

– **Erasure Tolerance.** A significant advantage of EvH over conventional block ciphers, e.g., AES-CBC, is erasure tolerance. The loss of a single ciphertext block in a mode like AES-CBC results in catastrophic failure, rendering all subsequent blocks undecipherable. EvH, in contrast, exhibits graceful degradation. The loss of data increases the receiver's computational workload but does not necessarily prevent the recovery of the entire message.
– **Message Length Concealment and Elasticity.** Traditional block cipher encryption schemes often produce a ciphertext whose length is directly proportional to the plaintext's length, potentially leaking valuable metadata to an adversary. EvH offers a degree of elasticity that can mitigate this. To achieve length concealment, the sender and receiver can negotiate a variable chunk size, cs. The Bloom filter's dimensions are then determined by the *count of inserted chunks*, rather than the raw byte length of the message. For example, a 100-character message

[6] It may happen in EvH that two different messages will result in different numbers of ones in Bloom filters, e.g., m_1 is 'A' and m_2 is 'Alice talks with Bob.' Here, the number of 1s will reveal the message length. To prevent this leakage, the sender can buffer a few messages and send them together.

encoded with $cs = 1$ results in 100 items inserted into the filter. Similarly, a 300-character message encoded with $cs = 3$ also results in 100 items (one prefix hash per 3-character chunk). Consequently, both messages can be encrypted into Bloom filters of identical size, concealing the true plaintext length from an observer. However, this flexibility involves a trade-off: using larger chunks reduces the granularity of the prefix search, potentially affecting the efficiency of online compression.

Further, a fundamental aspect of EvH's elasticity is its native ability to process messages of arbitrary length. Unlike block ciphers such as AES, which may necessitate padding inputs to align with block boundaries, EvH processes the message stream directly. This characteristic obviates the need for padding schemes, thereby eliminating both the associated data overhead and the attack vector for padding oracle vulnerabilities [28,23]. Furthermore, by decoupling the ciphertext structure from rigid block sizes, EvH mitigates side-channel leakage related to exact ciphertext length, often exploited in traffic analysis attacks [12].

– **Multi-Message Compression.** A unique capability of EvH compared to the block cipher methods is the ability to encrypt multiple, independent messages into a single Bloom filter by inserting the prefix sets of all messages into a Bloom Filter, (as has also been discussed in §5.1). A receiver, possessing the key and the specific IV for a desired message, can then perform the standard decryption search to retrieve it, completely agnostic to the other messages stored within the same ciphertext.
– **Decryption-Time Compression via Search Pruning.** As discussed in §5.1, the search-based nature of EvH decryption offers compression/decompression that occurs simultaneously with encryption/decryption, by choosing a higher false-positive rate (α) for the Bloom filter.

6 μ-Grams in EvH

Objective and high-level idea. In the worst case, recall that, Decryption Algorithm 2 needs to check all possible characters to find continuation. Now, our objective is to reduce the set of characters to be checked at each step of the decryption process; refer to Line 8 of Algorithm 2, thereby making the decryption algorithm efficient.

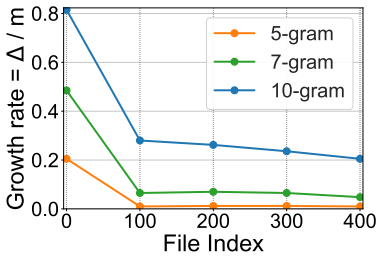
While keeping the alphabet unchanged, we introduce a set of μ-letter-grams that appear in the messages, allowing the decryption algorithm to disqualify a subset of characters at each step as illegal continuation paths. For instance, for a message "fun," $\mu = 2$-grams could include, "fu, un," and now the encryption algorithm will encrypt such μ-grams with the message into a Bloom filter, and the decryption algorithm will decrypt μ-grams and, based on them, decrypt the message. Now, the receiver maintains a dictionary of all valid μ-grams for a given language. Before checking a candidate prefix against the Bloom filter, the receiver first verifies that the μ-gram at the end of the prefix is valid or not. If not, the path is pruned without a cryptographic operation/$\mathsf{check}()$, and this dramatically reduces the search space.

This reduces $r = \alpha\sigma$, due to reducing the effective alphabet size σ (refer to §4.3 to see $r = \alpha\sigma$) and allows us to use a higher α, thereby allowing for compression, *i.e.*, space efficiency (see Figure 1b).

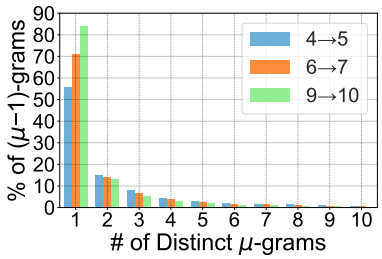
6.1 Encryption & Decryption using μ-grams

Encryption using μ-grams. When a sender and a receiver decide to exchange messages using EvH with μ-grams, they agree on k and an initial (possibly) empty set of μ-grams.

Each participant maintains their own copy of the set of μ-grams. Once the sender composes a message, she collects Δ — all the μ-grams that appear in the message (and not in the set),



(a) Exp 6.2.1: Normalized μ-gram-set growth for various $\boldsymbol{\mu}$'s.



(b) Exp 6.2.2: Prefix $\boldsymbol{\mu - 1}$ fanout percentage of total $\boldsymbol{\mu - 1}$ prefixes.

Fig. 2: Fanout and growth with different $\boldsymbol{\mu}$'s.

encrypts Δ into a Bloom filter using Algorithm 1. Then, the sender encrypts the message into the Bloom filter also.[7]

Decryption using μ-Grams. When the receiver obtains the Bloom filter, she first decrypts μ-gram set, *i.e.*, Δ, from the Bloom filter using Algorithm 2, adds the new μ-grams to their set, and then continues to decrypt the message.

Now, instead of checking each character in the Bloom filter (refer to Line 8-Line 9 of Algorithm 2), the receiver only checks for each of the μ-gram suffixes. If an i-th μ-gram suffix with the current prefix does not appear in the Bloom filter, we disqualify the i-th μ-gram for this prefix and continue the process with $(i+1)$-th μ-gram.

Example 6.1. Consider the message "apple is red" and let $k = 3$. The set of 3-grams extracted from the message is $x_1 = \text{app}, x_2 = \text{ppl}, x_3 = \text{ple}, x_4 = \text{le\_}, x_5 = \text{e\_i}, x_6 = \text{\_is}, x_7 = \text{is\_}, x_8 = \text{s\_r}, x_9 = \texttt{\_re}, x_{10} = \texttt{red}$, where _ denotes the space character. All these 3-grams form Δ.

The sender first encrypts all elements of the 3-gram set into a Bloom filter using Algorithm 1, then encrypts the entire message. Note that the Bloom filter encrypts both the valid μ-grams of the message and the message in the form of all prefix hashes.

The decryption begins using Algorithm 2 by first decrypting all 3-grams to form the set Δ, which is empty at this stage, and then decrypting the message with the help of Δ. To decrypt the message, Algorithm 2 checks $H(\text{a}), \ldots, H(\text{z})$ against the Bloom filter. Suppose only $H(\text{a})$ returns true, then the prefix "a" is inserted into the BFS queue. From this prefix, the receiver does not test all possible elements of Δ. Since $x_1 = \text{app}$ is the only 3-gram starting with "a," the receiver checks $H(\text{app})$, which succeeds and is added to the queue. The receiver then extends this prefix "app" by concatenating each 3-gram x_i and checks the bit of the Bloom filter corresponding to $H(\text{app} \parallel x_i)$. In this example, only $H(\text{apple\_})$ evaluates to true, due to $x_4 = \text{le\_}$. This process continues until the full message length is reached, at which point the recovered message "apple is red" is returned, and then the postamle characters are checked. □

Note on security of μ-grams. While this method works efficiently for high-entropy inputs, low-entropy inputs (e.g., aaaaa vs abcde with $\mu = 2$) require additional protection. For such cases, we randomly insert fake μ-grams into the Bloom filter. This ensures that, even when the message has repeating patterns, an adversary observing the Bloom filter sees multiple bits to be one, but cannot reliably distinguish which and/or how many μ-gram was encrypted.

[7] A single Bloom filter may include both the message and Δ; or two different Bloom filters can also be created.

6.2 Experimental Results

In this subsection, we experimentally study the behavior of μ-grams in real-world data and analyze how their growth impacts the communication overhead due to Δ and the decryption complexity. We investigate:

1. The growth behavior of newly introduced μ-grams as the database size increases, in Experiment 6.2.1.
2. The fanout distribution of $(\mu-1)$-grams and its implications for decryption branching, in Experiment 6.2.2.
3. Impact of different μ-grams on different parameters, such as growth rate, effective σ, and size of the Bloom filter, in Experiment 6.2.3.

Dataset and Methodology. We downloaded 200MB of plaintext from Wikipedia and partitioned it into 400 consecutive chunks of size 500KB. The data was processed incrementally to observe how the set of distinct μ-grams evolves as more data is added.

For each chunk, all μ-grams were generated using a sliding window. Let δ denote the number of *new* μ-grams introduced by a chunk, *i.e.*, μ-grams that did not appear in any previous chunk. To quantify the growth of the μ-gram set, we define the *growth rate* as:

$$\text{Growth Rate} = \delta/m$$

Here, m is the chunk size in bytes. This growth rate captures the fraction of newly observed μ-grams per byte of input data.

Although the complete μ-gram set can be stored locally using space-efficient data structures such as tries, the encryption process must transmit the explicit list of newly introduced μ-grams. Since each μ-gram is of μ bytes, the communication overhead for a chunk is $\Delta = \mu \times \delta$ bytes.

Experiment 6.2.1: Growth behavior of μ-grams. Figure 2a shows the growth rate of μ-grams as a function of the processed data size for $\mu \in \{5, 7, 10\}$. The x-axis shows the chunk index, which corresponds to the cumulative growth of the database in increments of 500KB. The y-axis shows the growth rate δ/m, *i.e.*, the number of newly introduced μ-grams per byte of input data.

At the beginning, the growth rate is high, since most encountered μ-grams are new. As more data is processed, the growth rate rapidly decreases and stabilizes. After approximately 100 chunks (corresponding to 50MB of data), the growth rate converges to an almost constant value for all different values of μ. This behavior indicates that the μ-gram set becomes saturated, and additional data introduces relatively few new μ-grams. Thus, since the communication overhead for a chunk $\Delta = \mu \times \delta$ becomes almost a constant after processing 50MB of data.

Experiment 6.2.2: Fanout of $(\mu-1)$-grams. To understand the effective branching factor during decryption, we analyze the fanout of $(\mu-1)$-grams after constructing the complete μ-gram set. Let γ denote a distinct $(\mu-1)$-gram. We examine how many distinct μ-grams have γ as their prefix. The number of such μ-grams is defined as the *fanout* of γ.

Figure 2b shows the distribution of fanout values for different choices of μ. The x-axis shows the fanout value, *i.e.*, the number of distinct μ-grams that share the same $(\mu-1)$-gram prefix. The y-axis shows the percentage of distinct $(\mu-1)$-grams whose fanout equals the corresponding x-axis value. Figure 2b shows that the majority of $(\mu-1)$-grams have very low fanout, mostly between 1 and 3. The fraction of $(\mu-1)$-grams with larger fanout values decreases rapidly, indicating that high branching is rare in practice. This shows that, in practice, the decryption process encounters limited branching.

Dict. Type	5-gram	7-gram	10-gram
Total % upto Fanout 30	99.3849	99.8541	99.9682
Asymptotic Set Growth Rate	0.0081	0.0462	0.2061
Δ Size ($\mu \times$ Growth Rate)	0.0405	0.3234	2.061
Valid/Effective Fanout (σ)	2.7936	1.7949	1.3293
Compression for $\alpha = (0.9/\sigma)$	0.3221	0.1792	0.1012
Size of Compressed File + Δ	0.3626	0.5026	2.1622

Table 2: Experiment 6.2.3: μ-gram statistics.

Experiment 6.2.3: μ-grams vs fanot, Δ, growth rate, compression, and size. Table 2 shows several metrics for μ-grams with different μ values: $5, 7, 10$. Each table row is discussed below.

- **Total % up to Fanout 30:** This row indicates the percentage of all $(\mu-1)$-grams whose fanout is less than or equal to 30. More than 99% of $(\mu-1)$-grams satisfy this condition for all μ values, meaning that most prefixes have a limited number of valid continuations. This confirms that the effective branching factor in the decryption process of EvH with μ-grams is small.
- **Asymptotic Set Growth Rate:** This row shows the growth rate of the μ-gram set as more data is processed. Lower values are better, as smaller growth reduces the number of new μ-grams (Δ) that need to be transmitted. Observe that the values increase with μ: 0.0081 for 5-grams, 0.0462 for 7-grams, and 0.2061 for 10-grams, showing that higher μ produces more new μ-grams per unit of data.
- **Δ Size ($\mu \times$ Growth Rate):** This row shows the size of new μ-grams (Δ) to be transmitted during encryption, computed by multiplying the growth rate by μ. Lower values are better to reduce communication overhead. As μ increases, the communication cost rises: 0.0405 for 5-grams, 0.3234 for 7-grams, and 2.061 for 10-grams.
- **Valid/Effective Fanout (σ):** This row shows the average number of valid continuations per $(\mu-1)$-gram — *i.e.*, the branching factor during the decryption process. Lower values are better because fewer candidates per prefix reduce decryption effort. As μ increases, the valid fanout decreases — 2.7936 for 5-grams, 1.7949 for 7-grams, and 1.3293 for 10-grams.
- **Compression for $\alpha = (0.9/\sigma)$:** This row shows the achievable compression factor of the Bloom filter when the parameter α is set inversely proportional to the valid fanout σ. In other words, a smaller σ (fewer valid continuations per prefix) allows the choice of a higher α, which reduces the false positive probability while keeping the Bloom filter size small. Lower values indicate better compression, meaning the Bloom filter occupies less space than the original message. The compression factors decrease as μ increases and σ decreases: 0.3221 for 5-grams, 0.1792 for 7-grams, and 0.1012 for 10-grams. This shows that higher μ-grams, which have smaller branching factors, allow us better compression of the Bloom filter without increasing false positives.
- **Size of Compressed File + Δ.** This row shows the total size of the transmitted encrypted message, including the compressed Bloom filter and the new μ-grams (Δ). Lower values are preferable to minimize transmission cost. As μ increases, it also increases Δ, and hence, the overall size to be transmitted increases.

Key Takeaways: (*i*) Most $(\mu-1)$-grams have low fanout, ensuring efficient decryption. (*ii*) Increasing μ raises the growth rate of Δ, and hence, increasing the communication cost. (*iii*) Larger μ reduces fanout, enabling higher α and more Bloom filter compression. (*iv*) The optimal μ requires balancing between Δ and Bloom filter false positive rate (and is related to the application and the data. Finding optimal μ is out of the scope of this paper).

7 Using LLMs in EvH

Objective and high-level idea. While the μ-Grams approach described in §6 improves the efficiency of the decryption algorithm, the μ-Grams approach incurs the overhead of maintaining

a dictionary of all possible μ-letter grams. Now, our objective is to eliminate this overhead by using a publicly available Large Language Model (LLM) to prune invalid prefixes encountered during the file decryption from the Bloom filter.

Particularly, the decryption algorithm will use a small local LLM (such as TinyLlama and Qwen-0.5B) to filter out prefixes that are unlikely to be legal. A prefix is considered legal if it can form either a meaningful English word or a sentence. The receiver queries the local small LLM to determine if a prefix is "semantically valid." Prefixes deemed nonsensical are pruned, again reducing the search space and enabling the use of smaller ciphertexts. We will demonstrate in §7.2 that using an LLM, the number of illegal prefixes parsed during file decryption decreases, thereby improving the performance.

7.1 LLM Prompt-based Prefix Validation

We design prompts to check whether a prefix forms a legal word or sentence during the decryption algorithm. We consider a prefix to be a legal word if it can form a semantically meaningful word on its own, or is likely to become a meaningful word with the addition of a few more characters at the end. E.g., a prefix 'ab' is considered legal as it can form a valid word, 'absent' or 'abacus,' whereas the prefix 'az' is not legal, as it is unlikely to lead to a valid English word.

Furthermore, if the candidate prefix consists of two or more legal words separated with space (*i.e.*, appears as a partial sentence), it is considered a legal sentence if it can either form a meaningful sentence on its own or be extended into one by appending additional words at the end. E.g., the prefix "I am" is legal since it can form a complete sentence as "I am at home," while a prefix like "I stood class" is invalid due to not being extended to form a coherent sentence.

Prompt 1 shows a legal word checking rule, and Prompt 2 shows a legal sentence checking rule to find meaningful English words and sentences that begin with the provided prefix. The prompt explicitly instructs LLM to assess the prefix based on the following criteria:

- Determine if the prefix is already a correctly spelled, recognized, and meaningful English word by itself (excluding abbreviations and misspellings).
- If the prefix is not a complete word, verify whether appending letters strictly at the end (without rearranging or altering the existing characters) can form a valid English word. This excludes forming abbreviations or inserting characters inside the prefix.
- Evaluate the prefix and decide if it constitutes a valid, meaningful English sentence.

LLM is instructed to respond in a structured format by producing only *true* or *false*, indicating whether the given prefix is valid.

Prompt 1: Meaningful Word Checking Prompt

You are an AI language analysis assistant. Your job is to find common, meaningful English words with a given prefix. Given a prefix, can you form a valid, common, meaningful English word?
Important Rule: A valid, common, meaningful English word must be correctly spelled and recognized as a standard word in English dictionaries, not merely a common misspelling or typo of another word.
To determine this:

1. Check if prefix is already a complete, correctly spelled, recognized, common, and meaningful English word on its own. Do not consider abbreviations.
2. If not, check if a correctly spelled, recognized, common, and meaningful English word can be formed by only appending new letters to the very end of the prefix, without changing, reorganizing, or inserting anything into the original prefix characters. Do not consider forming abbreviations.
3. Include inflected forms such as: - verbs with -ing, -ed, -s (*e.g.*, run → running, act → acting) - nouns with plural or suffixes (*e.g.*, act → action, act → actor)

Output as : result:true/false.

Prompt 2: Meaningful Sentence Checking Prompt

Can the string prefix form a valid, meaningful English word? This is possible only if the prefix is already a word. Your answer must be ONLY *true* or *false*. Do not add any other words, explanations, or punctuation to your response.

While using LLM for message/file decryption from Bloom filter, we consider four different scenarios, which assist in the pruning and efficient retrieval of files: (*i*) Prefix not present in Bloom filter, (*ii*) Prefix present in Bloom filter but LLM outputs invalid, (*iii*) Prefix present in Bloom filter and LLM outputs valid, and (*iv*) Prefix present in Bloom filter and LLM outputs invalid for true candidate prefix. Among these four cases, case (*i*) and case (*ii*) contribute to pruning illegal prefixes, while the remaining cases focus on identifying valid prefixes, enabling efficient file retrieval from the Bloom filter.

Due to the non-anticipatory nature of LLM, case (*iv*) may arise and cause file decryption from the Bloom filter to fail. For example, LLM may incorrectly label the legal prefix "ag" as *false*, although the lookup of "ag" returned 1 in the Bloom filter, and a valid word such as 'again' can be formed. To address this issue, the receiver interacts with the sender to ask for the missing part and, therefore, asks for the missing word(s) (*e.g.*, "again"). The missing word is communicated using another Bloom filter with a low false positive rate, α such as 10^{-2}, as described in Algorithm 1. The prompt is updated with the missing word retrieved from the Bloom filter (using Algorithm 2). For instance, the prompt is updated with the instruction as *prefix "ag" can be extended to form words such as "again"*.

7.2 Experimental Results

In this subsection, we experimentally validate how LLMs help us prune invalid suffixes.

Experiment 7.2.1: Fan-out rates with varying prefix size. We compare the fan-out rates (*i.e.*, the number of candidate prefixes) for varying prefix sizes observed during file retrieval from the Bloom filter, both with and without the support of LLM-based pruning. We store a file of size 1KB in a Bloom filter using the encryption method described in Algorithm 1. Without LLM support, the fan-out during file decryption grows exponentially, increasing from a fan-out of two candidate prefixes for a prefix length of one to ≈17K for a prefix length of ten. By adding LLM-based pruning support for file decryption via Bloom filters, we observe a significant decrease in the fan-out of candidate prefixes. Although LLM's behavior does not follow a defined pattern of prefix growth (similar to the trend observed in word length patterns in the English language), LLM avoids the exponential exhaustive search exhibited in EvH by pruning illegal prefixes, as depicted in Figure 3, which shows the size of prefix on x-axis and the y-axis shows the number of words scanned for the next iteration of Bloom filter search for a prefix of fixed size (using Algorithm 2).

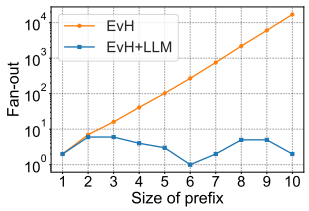


Fig. 3: Exp 7.2.1: Fan-out rates with varying prefix sizes.

8 Conclusion

This work introduced a new paradigm for symmetric encryption, demonstrating that by relaxing the traditional requirement for deterministic correctness, it is possible to design ciphers with powerful natively integrated functionalities (which at times may be useful). We presented EvH, the first cipher built on this paradigm, which trades a negligible, controllable probability of error for significant gains in erasure resilience and integrated compression.

References

1. Bellovin, S.M., et al.: Privacy-enhanced searches using encrypted bloom filters. IACR Cryptol. ePrint Arch. p. 22 (2004)
2. Ben-Efraim, A., et al.: Psimple: Practical multiparty maliciously-secure private set intersection. In: ASIA CCS. pp. 1098–1112 (2022)

3. Bernstein, D.J., et al.: Sphincs: Practical stateless hash-based signatures. In: EUROCRYPT. pp. 368–397 (2015)
4. Bloom, B.H.: Space/time trade-offs in hash coding with allowable errors. Communications of the ACM **13**(7), 422–426 (1970)
5. Bloom filter: https://en.wikipedia.org/wiki/Bloom_filter
6. Carbunar, B., et al.: Toward private joins on outsourced data. IEEE Trans. Knowl. Data Eng. **24**(9), 1699–1710 (2012)
7. Dauterman, E., et al.: DORY: an encrypted search system with distributed trust. In: OSDI. pp. 1101–1119 (2020)
8. Davidson, A., et al.: An efficient toolkit for computing private set operations. In: ACISP. pp. 261–278 (2017)
9. Dolev, S., et al.: HBSS: (simple) hash-based stateless signatures - hash all the way to the rescue! Cryptogr. Commun. **17**(3), 643–660 (2025)
10. Dong, C., et al.: When private set intersection meets big data: an efficient and scalable protocol. In: CCS. pp. 789–800 (2013)
11. Durham, E.A., et al.: Composite bloom filters for secure record linkage. IEEE Trans. Knowl. Data Eng. **26**(12), 2956–2968 (2014)
12. Dyer, K.P., et al.: Peek-a-boo, i still see you: Why efficient traffic analysis countermeasures fail. In: IEEE SP. pp. 332–346 (2012)
13. Egert, R., et al.: Privately computing set-union and set-intersection cardinality via bloom filters. In: ACISP. pp. 413–430 (2015)
14. Code available at https://tinyurl.com/57fe9a2k
15. Fu, Z., et al.: Toward efficient multi-keyword fuzzy search over encrypted outsourced data with accuracy improvement. IEEE Trans. Inf. Forensics Secur. **11**(12), 2706–2716 (2016)
16. Goh, E.: Secure indexes. IACR Cryptol. ePrint Arch. p. 216 (2003)
17. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: STOC. pp. 212–219 (1996)
18. Inbar, R., et al.: Efficient scalable multiparty private set-intersection via garbled bloom filters. In: SCN. pp. 235–252 (2018)
19. Kerschbaum, F.: Outsourced private set intersection using homomorphic encryption. In: ASIACCS. pp. 85–86 (2012)
20. Krell, F., et al.: Low-leakage secure search for boolean expressions. In: CT-RSA. pp. 397–413 (2017)
21. Li, R., et al.: Adaptively secure conjunctive query processing over encrypted data for cloud computing. In: ICDE. pp. 697–708 (2017)
22. Miyaji, A., et al.: A scalable multiparty private set intersection. In: International conference on network and system security. pp. 376–385 (2015)
23. Möller, B., et al.: This poodle bites: exploiting the ssl 3.0 fallback. In: Security Advisory (2014)
24. Pinkas, B., et al.: Faster private set intersection based on OT extension. In: USENIX Security Symposium. pp. 797–812 (2014)
25. Rindal, P., et al.: Improved private set intersection against malicious adversaries. In: EUROCRYPT. pp. 235–259 (2017)
26. Rivest, R.L., et al.: Chaffing and winnowing: Confidentiality without encryption. CryptoBytes (RSA laboratories) **4**(1), 12–17 (1998)
27. Vatsalan, D., et al.: Scalable privacy-preserving linking of multiple databases using counting bloom filters. In: ICDM Workshops. pp. 882–889 (2016)
28. Vaudenay, S.: Security flaws induced by cbc padding—applications to ssl, ipsec, wtls... In: EUROCRYPT. pp. 534–545 (2002)
29. Wang, B., et al.: Privacy-preserving multi-keyword fuzzy search over encrypted data in the cloud. In: INFOCOM. pp. 2112–2120 (2014)
30. Wu, M., et al.: Efficient unbalanced private set intersection cardinality and user-friendly privacy-preserving contact tracing. In: USENIX Security Symposium. pp. 283–300 (2023)
31. Zhang, E., et al.: Efficient multi-party private set intersection against malicious adversaries. In: CCSW. pp. 93–104 (2019)

A Security in Details

An adversary who possesses the ciphertext (Bloom filter) can perform membership queries for arbitrary prefixes. However, without the secret key, the adversary cannot compute PRF for any prefix, and thus cannot learn the encrypted message. The Bloom filter reveals only its bit pattern, which under a secure PRF is indistinguishable from a random bit string. This section formally proves the security of EvH encryption scheme.

A.1 Proof of Security

In this subsection, we prove the security of the EvH encryption scheme. We show that EvH achieves indistinguishability under a chosen-plaintext attack (IND-CPA) by reducing its security to the pseudorandomness of the underlying keyed hash function.

Theorem 2. *Let H be a secure pseudorandom function (PRF) family. Then the Encryption via Hash (EvH) scheme is IND-CPA secure.*

Proof. The proof proceeds in two main steps. First, we analyze the security of an idealized version of EvH where the PRF is replaced by a truly random function. Second, we show that if a computationally bounded adversary could distinguish encryptions in the real scheme (with the PRF), it could be used to construct a distinguisher that breaks the security of the PRF itself, which leads to a contradiction.

We define the IND-CPA security game for a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$:

1. **Setup:** The challenger generates a secret key K for the PRF H_K.
2. **Challenge:** $\mathcal{A}$ outputs a pair of equal-length messages, (m_0, m_1). The challenger chooses a random bit $b \in \{0, 1\}$, generates a fresh random Initialization Vector (IV), and computes the ciphertext. The ciphertext is the Bloom filter, denoted BF_b, which is populated by inserting the hash values $H_K(p_i)$ for every prefix p_i of the message $IV||m_b$. The challenger sends BF_b to $\mathcal{A}$.
3. **Guess:** $\mathcal{A}$ outputs a bit b'.

The advantage of adversary $\mathcal{A}$ is defined as:

$$Adv^{\text{IND-CPA}}_{\text{EvH},\mathcal{A}} = \left| \Pr[b' = b] - \frac{1}{2} \right|$$

The EvH scheme is IND-CPA secure if for any PPT adversary $\mathcal{A}$, this advantage is negligible.

Part 1: The Ideal World (Security with a Truly Random Function) Let's first consider an idealized scheme, EvH-Ideal, which is identical to EvH except that the PRF H_K is replaced by a truly random function $f : \{0, 1\}^* \to \{0, 1\}^\lambda$. In this ideal game, when the adversary $\mathcal{A}$ submits (m_0, m_1), the challenger selects a random bit b and a fresh random IV. The ciphertext BF_b is a Bloom filter populated based on the outputs of f. For each prefix p_i of the string IV$||m_b$, the value $y_i = f(p_i)$ is computed. These values $\{y_i\}$ are then used to set bits in the Bloom filter.

Crucially, because $m_0 \neq m_1$, the set of prefixes for IV$||m_0$ and IV$||m_1$ will differ. Let $P_0 = \{\text{prefixes of IV}||m_0\}$ and $P_1 = \{\text{prefixes of IV}||m_1\}$. For any prefix $p \in (P_0 \setminus P_1) \cup (P_1 \setminus P_0)$, the corresponding function output $f(p)$ is a fresh, uniformly random value. From the adversary's perspective, the resulting Bloom filters, BF_0 and BF_1, are populated by sets of

uniformly random and independent values (*i.e.*, for the given random function which generated BF_0 on IV$||m_0$ there is a random function that is drawn with same probability over the random functions space that would have generated the same value BF_0 on IV$||m_1$; similarly for BF_1) . An adversary $\mathcal{A}$ trying to guess b has no other information to leverage. Thus, its probability of success is exactly $\frac{1}{2}$, and its advantage is zero.

Part 2: The Reduction (From Real to Ideal) Now, we show that the security of the real EvH scheme can be reduced to the security of the PRF H. Assume for contradiction that there exists a PPT adversary $\mathcal{A}$ that can break the IND-CPA security of EvH with a non-negligible advantage ϵ. We will construct a distinguisher $\mathcal{D}$ that uses adversary $\mathcal{A}$ as a subroutine to break the PRF security of H. The distinguisher $\mathcal{D}$ is given access to an oracle $\mathcal{O}$ which is either the real PRF H_K (for a randomly chosen, unknown key K) or a truly random function f. $\mathcal{D}$'s goal is to determine whether $\mathcal{O}$ is H_K or f.

The distinguisher $\mathcal{D}$ works as follows:

1. **Simulate Challenger:** $\mathcal{D}$ starts running $\mathcal{A}$. When $\mathcal{A}$ outputs its challenge messages (m_0, m_1), $\mathcal{D}$ proceeds.
2. **Generate Ciphertext using Oracle:** $\mathcal{D}$ chooses a random bit $b \in \{0, 1\}$ and a random IV. It then constructs a ciphertext BF_b by simulating the EvH encryption for m_b. However, instead of using a known PRF key, $\mathcal{D}$ makes queries to its own oracle $\mathcal{O}$ for every prefix of IV$||m_b$. The outputs from $\mathcal{O}$ are used to populate the Bloom filter BF_b.
3. **Forward to Adversary:** $\mathcal{D}$ sends the generated ciphertext BF_b to $\mathcal{A}$.
4. **Receive Guess:** $\mathcal{A}$ outputs its guess b'.
5. **Output Decision:** If $b' = b$, $\mathcal{D}$ guesses that its oracle was the real PRF and outputs 1. Otherwise, $\mathcal{D}$ outputs 0.

Now, we analyze the advantage of our distinguisher $\mathcal{D}$.

- **Case 1: The oracle is the real PRF (H_K).** In this scenario, $\mathcal{D}$ provides $\mathcal{A}$ with a perfectly simulated environment of the real EvH IND-CPA game. By our assumption, $\mathcal{A}$ has advantage ϵ. The probability that $\mathcal{A}$'s guess is correct is: $\Pr[b' = b] = \frac{1}{2} + \epsilon$.
- **Case 2: The oracle is a truly random function (f).** In this scenario, $\mathcal{D}$ is simulating the EvH-Ideal game. As we argued in Part 1, $\mathcal{A}$ has no advantage. The probability that $\mathcal{A}$'s guess is correct is: $\Pr[b' = b] = \frac{1}{2}$.

The advantage of the distinguisher $\mathcal{D}$ in breaking the PRF security of H is:

$$\begin{aligned}\mathsf{Adv}_{\mathcal{D}}^{\text{PRF}} &= |\Pr[\mathcal{D} \text{ outputs } 1 \mid \mathcal{O} = H_K] - \Pr[\mathcal{D} \text{ outputs } 1 \mid \mathcal{O} = f]| \\ &= \left|\left(\frac{1}{2} + \epsilon\right) - \frac{1}{2}\right| \\ &= \epsilon.\end{aligned}$$

We have shown that if a PPT adversary $\mathcal{A}$ can break EvH with non-negligible advantage ϵ, then we can construct a PPT distinguisher $\mathcal{D}$ that breaks the underlying PRF security with the same advantage ϵ. This contradicts our initial assumption that H is a secure PRF. Therefore, no such adversary $\mathcal{A}$ can exist, and the EvH scheme is IND-CPA secure.

Next, note that this non-negligible advantage ϵ can be amplified to create a distinguisher that succeeds with overwhelming probability. The distinguisher $\mathcal{D}$ can run the adversary $\mathcal{A}$ for a polynomial number of times, p, each time providing it with a new challenge on a fresh pair of messages. $\mathcal{D}$ will then count the total number of $\mathcal{A}$'s successful guesses. If the oracle $\mathcal{O}$ is a truly random function, the number of successes will be concentrated around $p/2$. Conversely, if the

oracle is the real PRF H_K, the number of successes will be concentrated around $p{\cdot}(1/2{+}\epsilon)$. For a sufficiently large p, standard statistical concentration results, such as the Chernoff bound, show that these two outcome distributions are essentially disjoint with very high probability (their intersection has a negligible probability). This allows $\mathcal{D}$ to determine the nature of its oracle not just with a slight advantage, but with a probability arbitrarily close to 1, thus completing the reduction and claiming that if the encryption method is not semantically secure we have a test that refutes the pseudorandomality of the function family H.

A.2 Post-Quantum Security Context

While the security of EvH relies on hash functions like SHA-256, which are considered quantum-resistant, it is important to contextualize this claim. For symmetric-key primitives, the primary quantum threat is Grover's algorithm [17], which can reduce the effective security of an n-bit key to $n/2$ bits. The standard mitigation is to double the key size ($e.g.$, using AES-256 for 128-bit post-quantum security).

The advantage of EvH's hash-based construction is therefore not in offering a fundamentally different defense against Grover's algorithm, but in its structural properties. EvH's security is directly reducible to the properties of its underlying hash function.[8] Their relatively simple, non-algebraic structure is often considered less susceptible to unforeseen attacks or the insertion of backdoors compared to the complex algebraic structure of block ciphers. Furthermore, while SHA-256 is the reference implementation, EvH can be instantiated with any secure cryptographic hash function. For higher post-quantum security margins against attacks like Grover's algorithm, one could readily use SHA-384 or SHA-512.

[8] Once an ordered set of hash functions is coordinated among the encrypter and the decrypter, each message prefix can be encrypted (mapped into the Bloom filter) by a possibly different (even distinct) hash function from the set.